Multi-Source Candidate Data Transformer

Siddharth Kumar · siddharth.kumar_ug23@nsut.ac.in

Introduction

Candidate information often comes from multiple sources such as recruiter databases and resumes. These sources may use different formats, contain conflicting information, or have missing values. The goal of this project is to combine all of it into one clean and trustworthy Canonical Profile. To do that I first turn each source into small uniform facts (Claims) in a common format, decide whether records belong to the same person (Identity Resolution), resolve conflicts with a defined merge policy, and track where every final value came from (provenance) along with how confident the system is. If the system is not confident about a value, it leaves it empty instead of guessing.

Pipeline

- **Read input sources** (CSV, Resume PDF)
- **Extract candidate information** (Parse to Claims)
- **Standardize data** (Normalization)
- **Match candidate records** (Identity Resolution)
- **Merge into Canonical Profile**
- **Assign confidence and provenance**
- **Generate required output** (Config Projection)
- **Validate final JSON** (Schema Validation)

Canonical profile (output shape)

Field	Shape / type
candidate_id	string
full_name	string
emails	string[ ]
phones	string[ ] (E.164)
location	{ city, region, country }
links	{ linkedin, github, portfolio, other[ ] }
headline	string or null
years_experience	number or null
skills	[{ name, confidence, sources[ ] }]
experience	[{ company, title, start, end, summary }]
education	[{ institution, degree, field, end_year }]
provenance	[{ field, source, method }]
overall_confidence	number

Normalized formats

Field	Normalized to
Phones	E.164, e.g. +91 98765 43210
Dates	year-month, YYYY-MM (“present” kept)
Country	ISO two-letter code
Skills	canonical names (js, k8s ...)
Company	display kept; match-key used internally

Merge, conflict resolution and confidence assignment

- Records merge only with enough evidence they are the same person: a shared email or phone is strong evidence, while a matching name alone never triggers a merge.
- To avoid comparing every record with every other, similar candidates are first grouped by shared identifiers (email, phone, name, company) and only records in the same group are compared, which keeps the pipeline scalable.
- On conflicting values, the system keeps the one with the strongest evidence (source reliability times parser confidence); ties are broken consistently, so the same input always gives the same output.
- Collection fields (emails, phones, skills) are merged by combining unique normalized values.
- Two experience entries merge only when it is the same employer and the dates line up, so separate stints at one company stay separate.
- Every chosen value gets a confidence score and is linked to its source (provenance): agreement raises confidence, conflict lowers it, and rejected values are kept for traceability.

Configurable output and validation

- The pipeline always builds one complete Canonical Profile internally; a separate projection layer produces the output a user or application asked for, without changing the core pipeline.
- The config can choose which fields to include, rename fields, apply field-specific formatting, show or hide confidence and provenance, and set how missing values are handled (null, omit, or error).
- The result is validated against the requested schema before it is returned, so config mistakes are caught immediately rather than downstream.

Edge cases handled

- Location names are resolved carefully: "Karnataka" maps to India, while "CA" is read as California, not Canada.
- Company variations like "Google" and "Google LLC" are treated as one employer, but separate employment periods stay as different experience entries.
- Candidates with the same name are not merged on the name alone; they stay separate unless email or phone confirms the match.
- Missing, malformed or corrupt input files do not stop the run: invalid data is skipped, uncertain values are left empty, and the rest is still processed.
- The pipeline is deterministic (same input, same output), which is verified by a golden-output test.

Future improvements (intentionally left out)

- OCR or ML-based resume parsing for scanned or highly unstructured resumes.
- More input sources such as LinkedIn, GitHub, or ATS JSON.
- Larger country and skill dictionaries for broader normalization coverage.